LAPORAN PRAKTIKUM
INTERNET OF THINGS II



Disusun Oleh:

Nama: M Riki Pratama
Kelas: Teknologi Rekayasa Internet (3C)
NPM: 25782072

Dosen:
Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG 2026

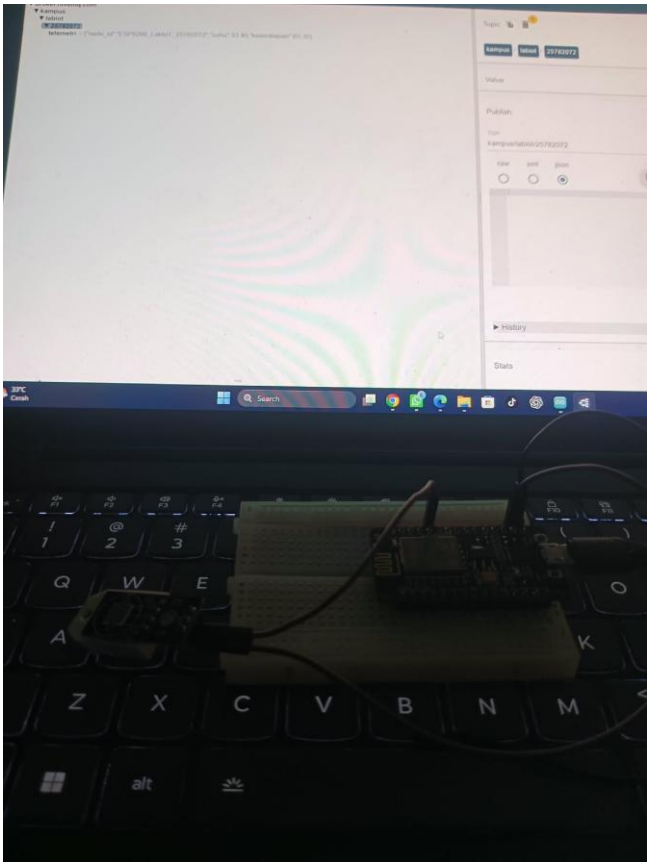
6.5 Praktikum

6.5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Publisher*. Perangkat membaca data suhu dan kelembapan dari sensor DHT11/DHT22, mengemas parameter tersebut ke dalam dokumen JSON menggunakan ArduinoJson, lalu memublikasikannya setiap 5 detik ke broker publik broker.hivemq.com.

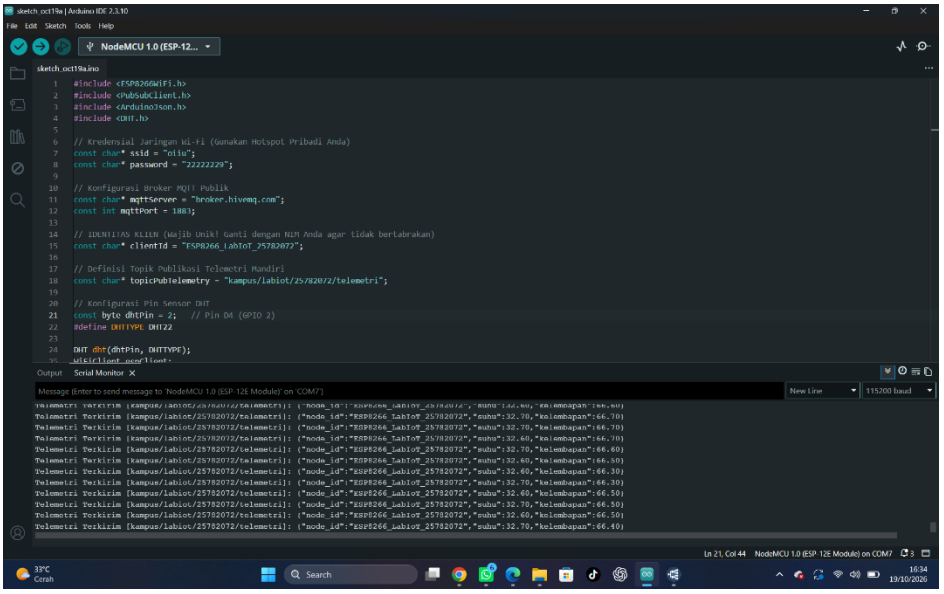
Langkah Kerja Rangkaian:

1. Pasang NodeMCU ESP8266 pada *breadboard*.
2. Sensor DHT11/DHT22: Hubungkan pin VCC ke 3V3, pin GND ke GND, dan pin Data ke D4 (GPIO 2).



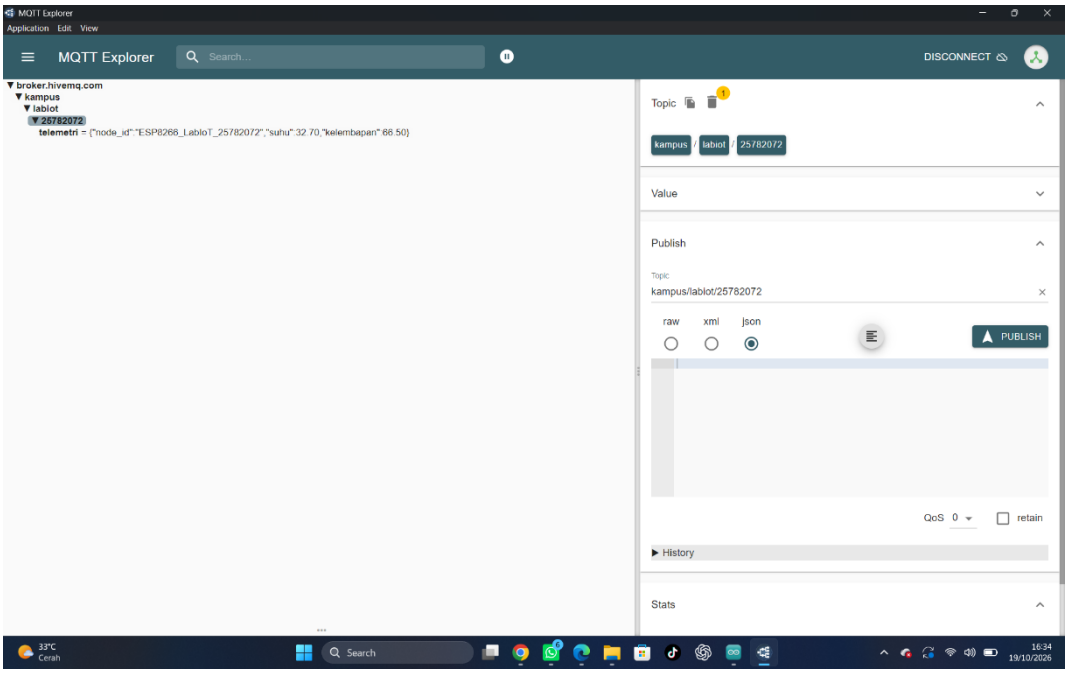
Langkah Kerja Pemrograman:

1. Buka Arduino IDE, buat berkas sketsa baru (*File* > *New Sketch*).
2. Tuliskan kode program berikut:



3. Ganti NAMA_WIFI_ANDA, PASSWORD_WIFI_ANDA, clientId (gunakan NIM Anda), dan topik kampus/labiot/nim_anda/telemetri dengan NIM pribadi Anda.
4. Unggah (*upload*) program ke NodeMCU ESP8266, buka Serial Monitor pada *baud rate* 115200.
5. Amati log: ESP8266 akan terhubung ke Wi-Fi, melakukan jabat tangan dengan broker broker.hivemq.com, dan memublikasikan paket JSON setiap 5 detik.

Output:

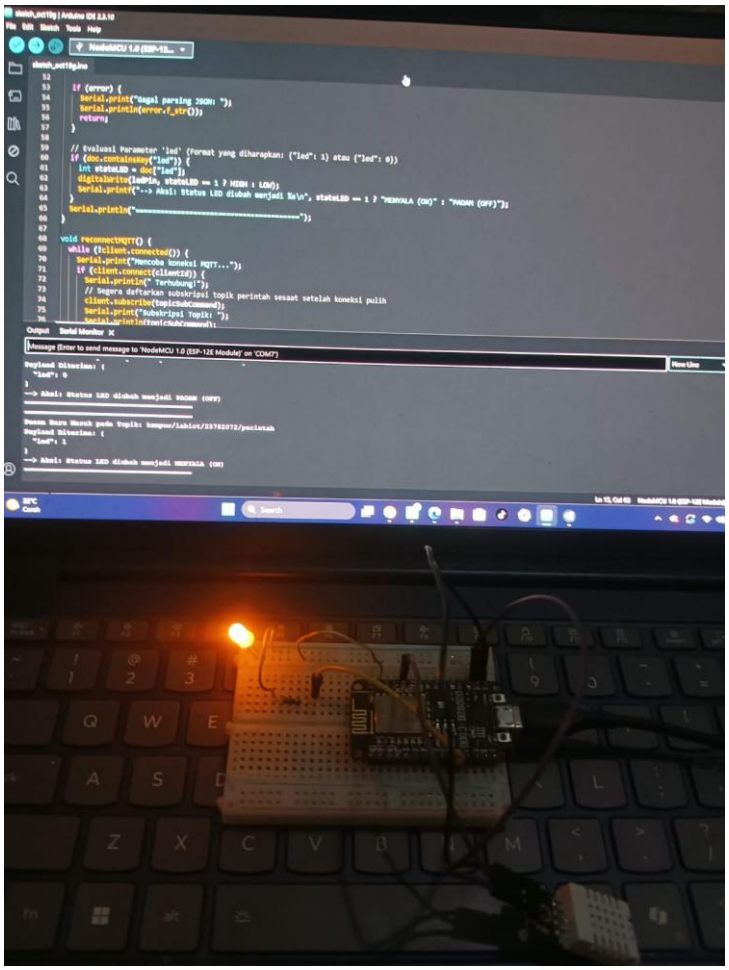


6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Subscriber*. Mikrokontroler mendaftarkan minatnya pada topik perintah (kampus/labiot/nim_anda/perintah). Saat pengguna luar (melalui aplikasi penguji MQTT) memublikasikan perintah berformat JSON, fungsi *callback* NodeMCU akan membedah payload dan menyalakan/mematikan lampu LED indikator secara seketika.

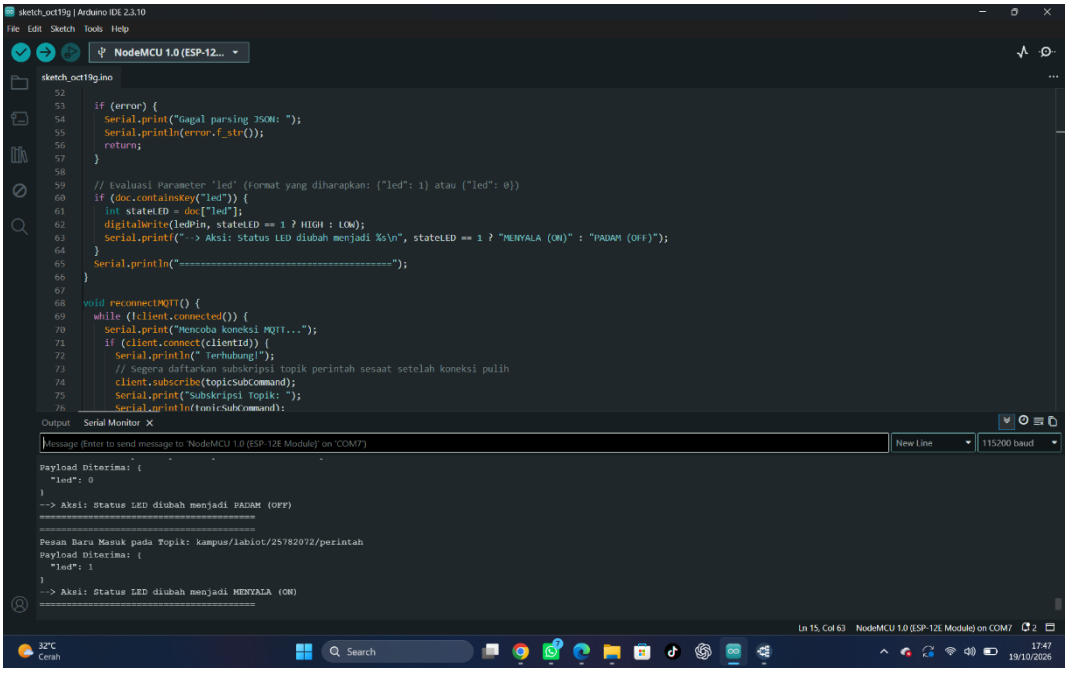
Langkah Kerja Rangkaian:

1. Pertahankan sensor DHT pada pin D4 dari Praktikum 1.
2. LED Indikator: Hubungkan kaki Anoda (kaki panjang) LED ke pin D1 (GPIO 5), dan hubungkan kaki Katoda (kaki pendek) ke Resistor 220 Ωyang menuju ke GND.



Langkah Kerja Pemrograman:

1. Buat sketsa baru di Arduino IDE, lalu masukkan kode berikut:

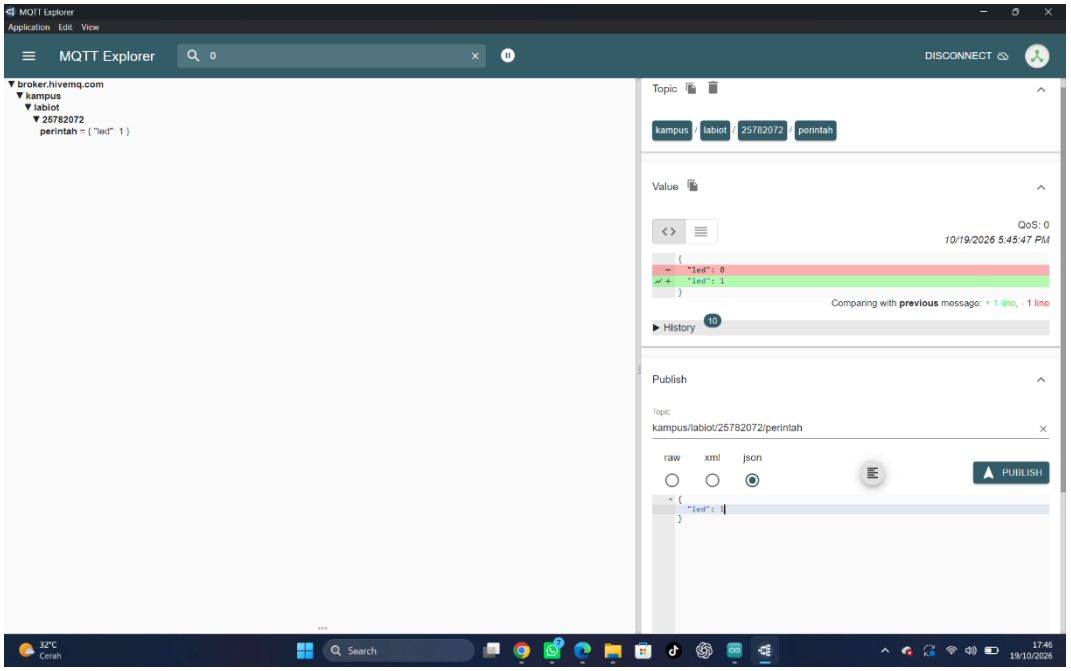


2. Unggah kode ke board NodeMCU Anda.

Penjelasan Singkat Kode:

- WiFiClient espClient; : Membuka antarmuka soket jaringan TCP level dasar pada cip ESP8266.
- PubSubClient client(espClient); : Menginisialisasi objek protokol MQTT yang beroperasi di atas soket jaringan espClient.
- client.setServer(mqttServer, mqttPort); : Menetapkan alamat host broker dan port layanan (1883 merupakan port standar MQTT teks terbuka / tanpa TLS).
- client.loop(); : Fungsi wajib. Rutinitas ini bertugas memproses paket TCP masuk, menjaga interval *keep-alive ping* dengan server agar koneksi tidak diputus paksa (*timeout*), serta mengelola antrean pesan.
- StaticJsonDocument<256> doc; : Mengalokasikan blok memori tetap di SRAM (sebesar 256 byte) untuk menampung pohon objek JSON tanpa memicu fragmentasi memori *heap*.
- serializeJson(doc, jsonBuffer); : Mengubah pohon objek JSON menjadi representasi string karakter padat yang siap ditransmisikan.

Output:



6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

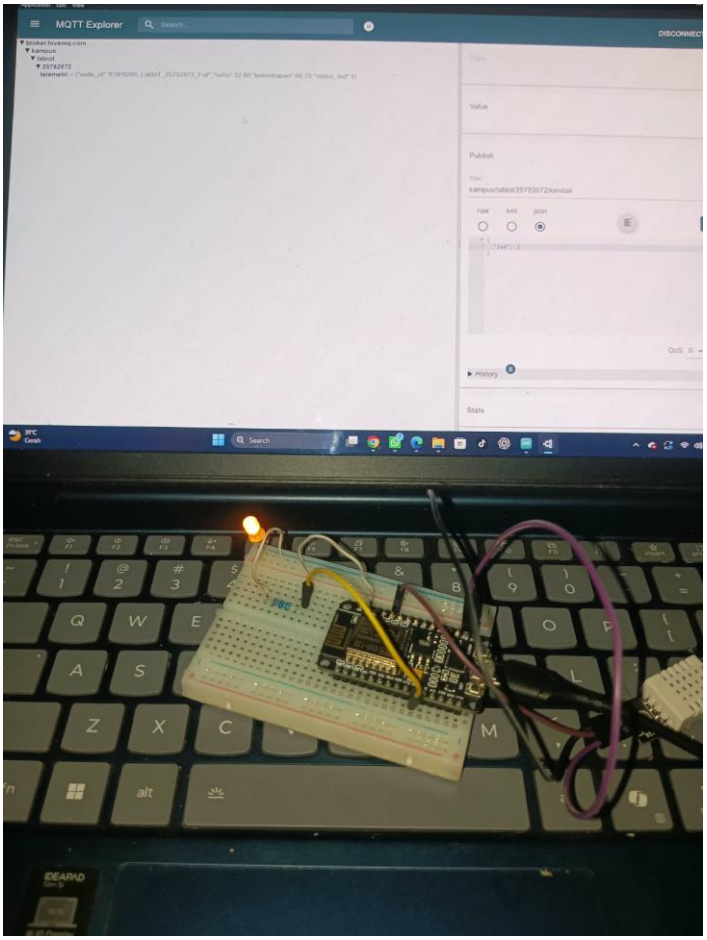
Pada praktikum ini, kita menyatukan kapabilitas Praktikum 1 dan Praktikum 2 ke dalam sebuah sistem terpadu (*Full-Duplex Telemetry & Control*). NodeMCU secara kontinu memublikasikan data telemetri sensor lingkungan (Suhu, Kelembapan, dan Status LED fisik) ke Broker setiap 4 detik, sembari tetap responsif mendengarkan dan mengeksekusi instruksi kontrol LED dari topik perintah kapan saja.

1. Langkah Kerja Rangkaian (Hardware)

Untuk merangkai sistem terpadu dua arah (menggabungkan pembacaan sensor dan kendali LED), siapkan komponen dan hubungkan sesuai instruksi berikut:

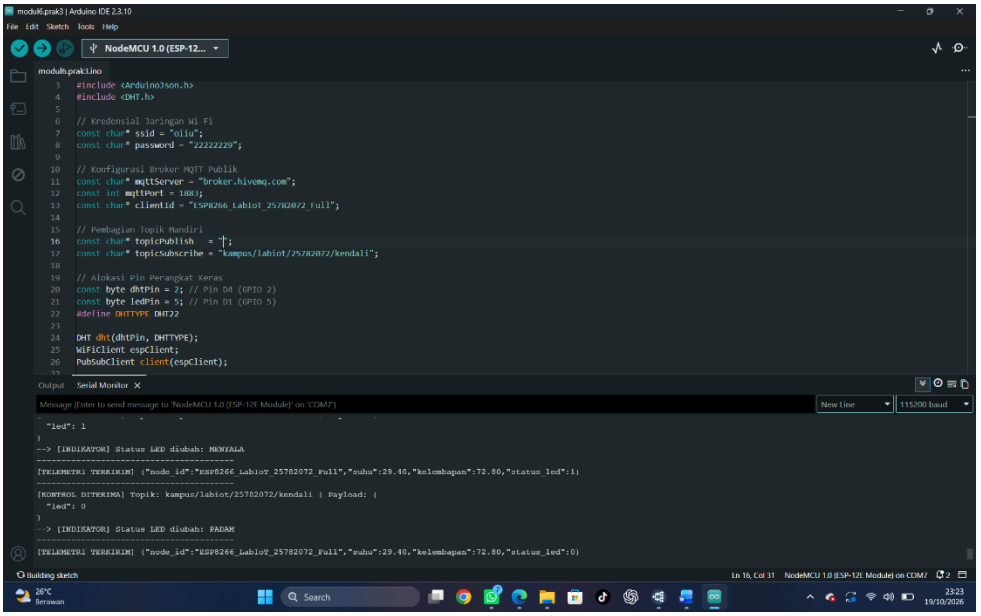
- **Sensor Suhu & Kelembapan (DHT11 / DHT22):**
  - Hubungkan pin **VCC** ke pin 3V3 pada NodeMCU ESP8266.
  - Hubungkan pin **GND** ke pin GND.
  - Hubungkan pin **Data** ke pin **D4 (GPIO 2)**.
- **Indikator LED:**
  - Hubungkan kaki Anoda (kaki panjang yang positif) ke pin **D1 (GPIO 5)** pada NodeMCU ESP8266.
  - Hubungkan kaki Katoda (kaki pendek yang negatif) melalui Resistor 220 Ω menuju ke pin GND.
- **Koneksi USB:** Hubungkan NodeMCU ESP8266 ke komputer/laptop menggunakan kabel data Micro-USB.

:



Langkah Kerja Pemrograman:

1. Masukkan kode terpadu berikut ke Arduino IDE:

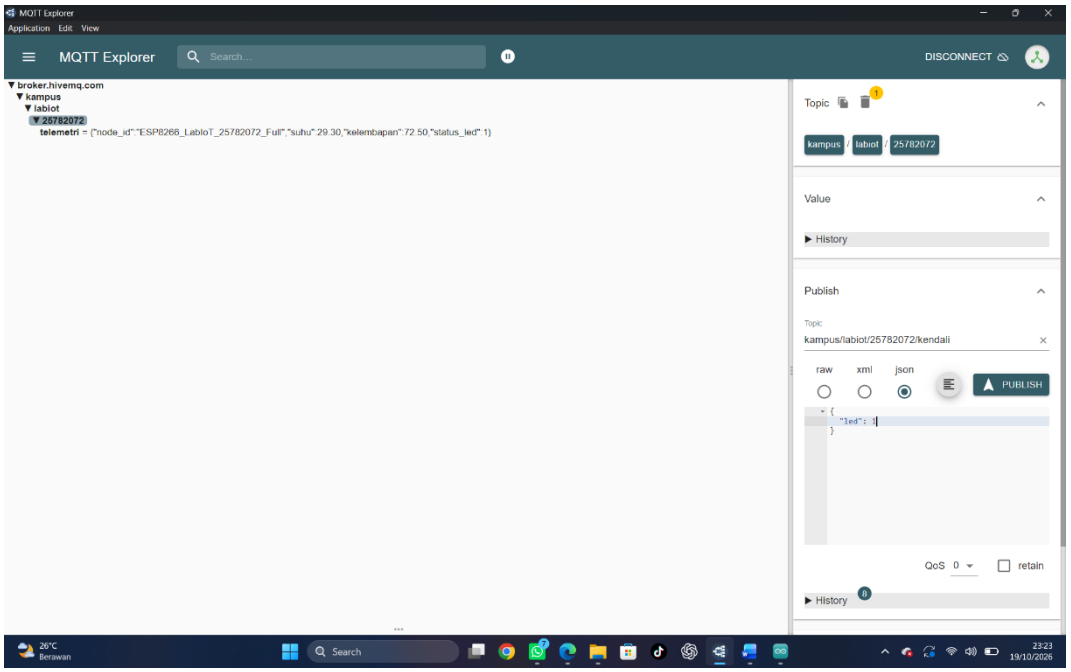


2. Unggah kode ke NodeMCU ESP8266 dan biarkan sistem berjalan.

3. Langkah Pengunggahan & Pengujian

1. Buka Arduino IDE, pastikan board yang dipilih adalah NodeMCU 1.0 (ESP-12E Module) dan pilih *Port* USB yang sesuai.
2. Unggah (*Upload*) program ke mikrokontroler.
3. Buka Serial Monitor pada *baud rate* 115200 untuk memantau proses koneksi Wi-Fi dan pengiriman data telemetri berkala setiap 4 detik.
4. Buka aplikasi MQTT Explorer, sambungkan ke broker broker.hivemq.com (Port 1883) Kemudian klik connect
5. Pantau data JSON yang masuk pada hierarki topik kampus/labiot/25782072/telemetri.
6. Uji kendali dua arah dengan mengirimkan payload JSON {"led": 1} atau {"led": 0} melalui topik kampus/labiot/25782072/kendali untuk menyalakan atau memadamkan LED secara asinkron.

Output:



6.6 Latihan

Kerjakan latihan berikut secara mandiri berbasis analisis teori dan modifikasi kode program untuk melengkapi laporan praktikum Anda:

- Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):
  Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:
  i. Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler. MQTT memiliki ukuran *header* yang sangat kecil (minimal hanya 2 byte), sedangkan HTTP/REST memiliki beban *header* (*overhead*) yang jauh lebih besar ($\approx 200 - 800$ byte), sehingga MQTT sangat hemat kapasitas data seluler.
  ii. Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah.
  Protokol MQTT menggunakan model *Publish-Subscribe* terpusat yang dikoordinasikan oleh broker tanpa perlu mengelola siklus jabat tangan *request-response* yang berat, sehingga meringankan beban pemrosesan pada mikrokontroler
  iii. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*).
  Distribusi Satu ke Banyak (*1-to-N Distribution*): Melalui arsitektur topiknya, sebuah pesan dari satu *publisher* dapat disebarkan secara instan oleh broker ke jutaan *subscriber* secara bersamaan tanpa membebani perangkat pengirim.

- Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):
  Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (+ atau #) untuk:
  i. Memantau parameter suhu dari seluruh mesin di semua lini produksi.
  pabrik/+/+/suhu (menggunakan *single-level wildcard* + untuk menggantikan variasi lini dan mesin).
  ii. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2.
  pabrik/line2/m1/# (menggunakan *multi-level wildcard* # di ujung

untuk menangkap semua sub-parameter sensor di bawah mesin tersebut).

iii. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.
pabrik/# (menangkap seluruh hierarki turunan di bawah domain pabrik).

- Latihan 3 (Implementasi Serialisasi Data Dinamis):
Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

```
// ============================================================
// MODIFIKASI LATIHAN 3: Penambahan Parameter Dinamis
// ============================================================
doc["uptime_detik"] = millis() / 1000;         // Waktu aktif sistem (detik)
doc["ip_lokal"] = WiFi.localIP().toString();   // IP lokal mikrokontroler
// ============================================================
```

i. "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).
Berfungsi untuk menghitung lama waktu aktif sistem mikrokontroler dalam satuan detik yang diperoleh dari pembagian nilai fungsi millis() / 1000

ii. "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).
Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!
Berfungsi untuk mengambil informasi alamat IP lokal yang didapatkan oleh modul ESP8266 dan mengubahnya ke dalam bentuk format teks string menggunakan WiFi.localIP().toString()

- Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):
Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().

i. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()?
Fungsi client.loop() akan berhenti berjalan selama 6 detik penuh. Akibatnya, mikrokontroler tidak akan mampu memproses paket data masuk dari broker, gagal merespons callback perintah kendali secara asinkron (*non-responsive*), dan berisiko memutus koneksi

jaringan karena gagal mengirimkan sinyal *keep-alive ping* tepat waktu.

ii. Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

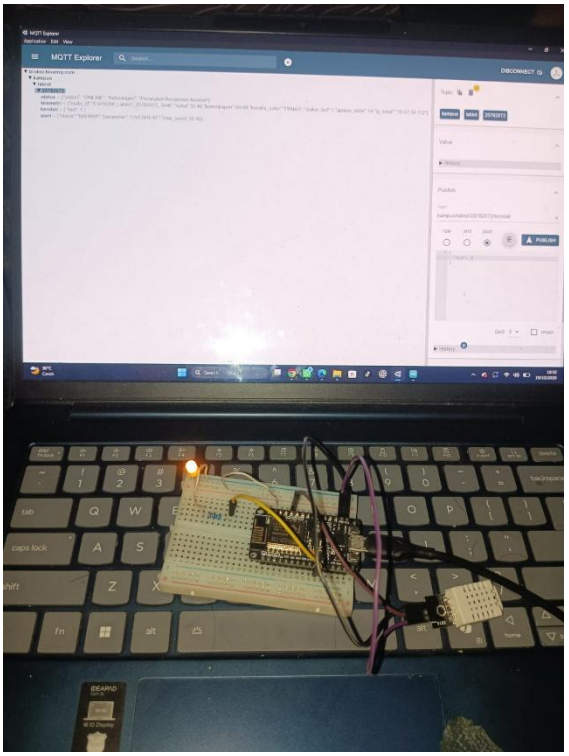
Fungsi reconnect() memantau status koneksi secara terus-menerus melalui perulangan while (!client.connected()). Jika koneksi terputus, program akan mencoba melakukan jabat tangan ulang ke broker menggunakan client.connect(clientId). Sesaat setelah status kembali terhubung (*succesfully connected*), fungsi ini langsung mendaftarkan ulang subskripsi topik (*re-subscribe*) agar perangkat dapat kembali mendengarkan perintah masuk tanpa kehilangan sinkronisasi.

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri

Kembangkan implementasi Praktikum 3 menjadi sistem pengawasan keselamatan dan proteksi otomasi industri tertutup (*Closed-Loop Industrial Safety & Automated Actuation*) yang tangguh dan cerdas secara mandiri:

- **Tujuan:** Mampu memadukan akuisisi sensor suhu dan kelembapan, serialisasi JSON dinamis berdasar kondisi lingkungan, subskripsi kendali indikator LED jarak jauh, kanal peringatan darurat independen (*dedicated alert topic*), dan pendaftaran pesan wasiat *Last Will and Testament* (LWT) secara mandiri.
- **Tingkat Kesulitan:** Lanjut (Mandiri / Individu)

Instruksi Perangkaian:



1. Pertahankan konfigurasi perangkat keras dari Praktikum 3:
    - Sensor DHT11/DHT22 terhubung ke pin **D4 (GPIO 2)** (VCC ke 3V3, GND ke GND).
    - LED Indikator + Resistor 220 Ωterhubung ke pin **D1 (GPIO 5)** (Katoda resistor ke GND).

Instruksi Pemrograman (Rule Engine & Safety Logic):

1. **Klasifikasi Status Lingkungan pada Telemetri Rutin:**
   Modifikasi dokumen JSON telemetri rutin (kampus/labiot/nim_anda/telemetri) agar menambahkan kunci "kondisi_suhu" bertipe string:
    - JIKA suhu ≤ 31.0°C, isi nilai menjadi "NORMAL".
    - JIKA suhu > 31.0°C, isi nilai menjadi "PANAS".
2. **Kanal Peringatan Bahaya Mandiri (*Dedicated Alert Topic*):**
   Implementasikan logika pengawasan ambang batas kritis lokal (*edge safety engine*):
    - **JIKA** suhu terukur melebihi ambang batas 31.0°C:
        - Sistem harus segera memublikasikan paket peringatan bahaya ke topik khusus: kampus/labiot/nim_anda/alert dengan format JSON:

```
if (!alertSentState) {
  StaticJsonDocument<256> docAlert;
  docAlert["status"] = "BAHAYA";
  docAlert["parameter"] = "OVERHEAT";
  docAlert["nilai_suhu"] = serialized(String(t, 2));
```

- Secara otomatis menyalakan LED indikator peringatan bahaya tanpa menunggu perintah dari operator luar.
- Berikan proteksi penanda (*flag / state latch*) agar pesan peringatan ini tidak membanjiri broker setiap milidetik, melainkan dikirimkan teratur saat kondisi bahaya terdeteksi pertama kali.
-

3. Mekanisme **Last Will and Testament (LWT):**
   Konfigurasikan fungsi client.connect() dengan menyertakan parameter pendaftaran pesan wasiat (*Last Will*):
   - **Topik LWT:** kampus/labiot/nim_anda/status
   - **Pesan Wasiat LWT**:

```
"{\"status\": \"OFFLINE\", \"keterangan\": \"Perangkat Terputus Tak Terduga\"}"
```

- **QoS LWT:** 1
- **Retain Flag:** true
- Sesaat setelah mikrokontroler berhasil terhubung ke Broker, publikasikan pesan status aktif (*online message*) ke topik status yang sama:

```
"{\"status\": \"ONLINE\", \"keterangan\": \"Perangkat Beroperasi Normal\"}", true);
```

4. **Kendali Manual Override Saat Normal:**
   Saat suhu berada di bawah ambang batas bahaya, operator luar tetap dapat menyalakan atau mematikan LED secara manual melalui pengiriman payload {"led": 1} atau {"led": 0} ke topik kampus/labiot/nim_anda/kendali.

